not the point: you have complete control! Create the following class in your test project:

```
public class StubCurrencyConverter : ICurrencyConverter
{
    public decimal ConvertToGbp(decimal value, decimal rate, int dps)
    {
        return 3;
    }
}
```

You now have all the pieces you need to replace the implementation in your tests. To achieve that, we'll use a feature of `WebApplicationFactory` that lets you customize the DI container before starting the test server.

TIP It's important to remember that you want to replace the implementation only when running in the test project. I've seen some people try to configure their real apps to replace live services for fake services when a specific value is set, for example. That is often unnecessary, bloats your apps with test services, and generally adds confusion!

`WebApplicationFactory` exposes a method, `WithWebHostBuilder`, that allows you to customize your application before the in-memory `TestServer` starts. The following listing shows an integration test that uses this builder to replace the default `ICurrencyConverter` implementation with our test stub.

Listing 36.9 Replacing a dependency in a test using `WithWebHostBuilder`

```
public class IntegrationTests:                                          ❶
    IClassFixture<WebApplicationFactory<Startup>>                       ❶
{                                                                       ❶
    private readonly WebApplicationFactory<Startup> _fixture;           ❶
    public IntegrationTests(WebApplicationFactory<Startup> fixture)     ❶
```